

Preinforme – Desafío 1

Implementación de Sweet Crush mediante manipulación de bits

Sofía Arboleda Zapata
Jose Manuel Arias Toro

11 de septiembre de 2026

1. Contextualización

El presente proyecto abarca el análisis y diseño conceptual para la implementación del juego *Sweet Crush* en C++. El sistema administrará un tablero rectangular de dimensiones variables ($F \times C$), permitiendo visualizar fichas, detectar combinaciones, procesar eliminaciones y reorganizar el tablero mediante mecánicas de gravedad.

Bajo las restricciones técnicas del desafío, el tablero no empleará tipos de datos estándar (como `int` o `char`) por cada posición. En su lugar, cada ficha será representada con exactamente **3 bits**, permitiendo codificar hasta 8 estados diferentes: 6 para los tipos de fichas activas (representadas visualmente mediante caracteres ASCII: @, #, \$, %, &, +), un estado para celdas vacías (.) y un estado temporal de marcado (*) para eliminación. Por ende, la memoria dinámica se administrará como un bloque continuo de bytes (`unsigned char*`), obligando a manipular la información mediante operadores a nivel de bits (&, |, ^, ~, <<, >>).

2. Análisis del problema

El núcleo de la solución radica en la traducción entre la posición lógica del tablero (fila, columna) y el mapeo físico dentro del arreglo de bytes en el *Heap*.

1. **Cálculo de espacio físico:** Para F filas y C columnas, el número total de bits requeridos es $B = 3 \times F \times C$. La cantidad exacta de bytes N a reservar dinámicamente es $\lceil B/8 \rceil$.
2. **Indexación lógica y mapeo de bit:** Una celda (f, c) se convierte a un índice lineal:

$$\text{índice} = f \times C + c$$

El bit donde inicia la representación de dicha ficha dentro de la secuencia continua es $\text{bitInicial} = \text{índice} \times 3$.

3. **Ubicación del Byte y Desplazamiento:** La ficha se ubica a partir del byte $\lfloor \text{bitInicial}/8 \rfloor$, con un desplazamiento dado por $\text{bitInicial} \bmod 8$.

Debido a que cada celda ocupa 3 bits y los bytes contienen 8, una ficha puede estar contenida completamente dentro de un byte o quedar dividida entre dos bytes consecutivos. La extracción y modificación de cada ficha se realizará mediante una máscara de 3 bits con el valor 7 (111 en binario) y desplazamientos, garantizando la lectura y escritura sin alterar las celdas adyacentes.

3. Diseño de la solución

La arquitectura del proyecto adopta una estructura modular en C++ orientada a programación funcional (archivos `.h` y `.cpp`), manteniendo la separación estricta entre la manipulación de bajo nivel y la lógica del juego.

- `bits.h` / `bits.cpp`: Funciones de bajo nivel. Mapeo de índices, lectura/escritura de 3 bits mediante máscaras/desplazamientos y cálculo de requerimientos de memoria.
- `tablero.h` / `tablero.cpp`: Gestión del almacenamiento físico en el *Heap* (`new[]`/`delete[]`), generación aleatoria, vista de fichas y binaria y reestructuración por gravedad.
- `combinaciones.h` / `combinaciones.cpp`: funciones encargadas de buscar combinaciones horizontales y verticales, marcar las posiciones involucradas y procesar las eliminaciones y cascadas.
- `estructura.h` / `estructura.cpp`: funciones destinadas a modificar las dimensiones del tablero, incluyendo la adición y eliminación de filas y columnas, así como la reorganización de la memoria cuando sea necesario.
- `juego.h` / `juego.cpp`: funciones relacionadas con el control de los turnos, validación de acciones, estadísticas y puntuación.
- `main.cpp`: Punto de entrada, flujo principal y liberación controlada de recursos.

4. Esquema general de funcionamiento

El comportamiento global de la aplicación se ilustra en la Figura 1.

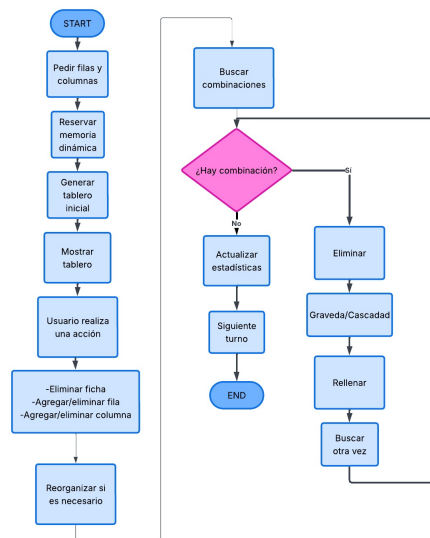


Figura 1: Diagrama de flujo general para la solución propuesta.

El proceso inicia con la solicitud de dimensiones y la reserva en memoria dinámica. Luego de cada acción del usuario (eliminar ficha o cambiar estructura del tablero), el sistema ingresa a un ciclo de evaluación de combinaciones. Si se detectan grupos válidos, las fichas marcadas se eliminan, se aplica gravedad vertical, se rellenan los vacíos y se reevalúa la búsqueda de forma iterativa hasta resolver todas las cascadas automáticas producidas.

5. Consideraciones de implementación

- **Gestión de Memoria y Redimensionamiento:** Las adiciones o eliminaciones de filas/columnas intermedias desplazan masivamente la posición física de los bits. Para optimizar el uso del *Heap*, las reasignaciones con `new[]` solo se ejecutarán cuando la ocupación física caiga por debajo del 65 % respecto a la capacidad reservada.

- **Evaluación sin estructuras auxiliares:** Como cada casilla usa 3 bits, tenemos 8 estados. Usamos los valores del 0 al 5 para las figuras, el 6 para casillas vacías y el 7 como una marca de borrado. Así sabemos qué fichas eliminar sin gastar memoria adicional.